

Algorithms for the Sum-check Protocol

2025-11-25 | [Quang Dao](#)

The Need for Verifiability

We increasingly rely on computational results that we cannot feasibly verify ourselves. Such verification barriers come in two forms. First, the computation may involve private data that cannot be shared, such as financial audits, medical studies, and proprietary algorithms. Second, even when data is public, re-executing the computation may be prohibitively expensive: large scientific analyses might require days of cluster time on terabytes of data. How can we trust that such computations were performed correctly?

Cryptographic proof systems solve this dilemma. Such protocols let an untrusted prover, who possesses some private data, convince a verifier that the data satisfies a public property. For example, this could include proving that you are over 18 or that your account has sufficient balance, without revealing anything else such as your full date of birth, or your financial history. Furthermore, the proofs or certificates produced by such systems are relatively short, and can be verified in a matter of milliseconds, rather than the potential hours or days required to re-execute the computation. This saving further compounds if there are **many** parties that need to verify the result.

Over the past decade, cryptographic proof systems have matured from theoretical curiosities into practical tools used for a variety of applications, including age verification [1] and blockchain scalability [2]. **To make developing proof systems for each application easier, the community is converging on a common framework: zero-knowledge virtual machines (zkVMs).**

A zkVM is a virtual computer that can run any program **compiled to a common instruction-set architecture like RISC-V** and produce a certificate proving that the execution is correct. This allows developers to write programs in high-level languages like Rust or Python, compile them to RISC-V, and prove their execution correctness, all without needing deep cryptographic expertise. As these systems see wider adoption, prover efficiency becomes paramount: even for the fastest zkVMs, generating a proof is still 50,000 to 100,000 times slower than running the computation natively. Teams across academia and industry are racing to close this gap.

Many state-of-the-art zkVMs, such as Jolt [3], owe much of their performance to a classical protocol from 1992: the sum-check protocol [4]. But sum-check's very success has made it a bottleneck—in Jolt, it consumes about 70% of the total proving time for large programs. In this blog post, I will introduce the sum-check protocol, detail well-known algorithms for its prover, and present a new technique that speeds up the prover in settings relevant to zkVMs, which I am currently integrating into Jolt.

Sum-Check Protocol Overview

The sum-check protocol is an interactive proof that allows an untrusted prover to convince a computationally limited verifier of the value of a very large sum. Informally, the verifier wants to check the **sum of a multivariate polynomial** over a large product domain, but would like to avoid **explicitly adding up all of the terms.**



Formally, we fix a finite field $\mathbb{F}$ and a multivariate polynomial $p(X_1, \dots, X_n) \in \mathbb{F}[X_1, \dots, X_n]$, of degree bounded by d in each variable. The sum-check claim is then

$$\sum_{x_1 \in H_1, \dots, x_n \in H_n} p(x_1, \dots, x_n) = c,$$

for some evaluation domains $H_1, \dots, H_n \subseteq \mathbb{F}$ and a claimed value $c \in \mathbb{F}$. In most applications, and for the remainder of this blog post, **we restrict to the Boolean hypercube**, which is the domain $H_1 = \dots = H_n = \{0, 1\}$.

The verifier knows p , or at least has oracle access to evaluations of p at points of its choice, but wants to use this query access as little as possible. Naively, the verifier could evaluate p on all 2^n points of $\{0, 1\}^n$ and sum the results, which takes work on the order of $O(2^n)$. The key idea of sum-check is that, by interacting with an untrusted prover who supplies additional “auxiliary” polynomials, the verifier can reduce the work of checking the original claim to checking a related claim about p at a single randomly chosen point.

In particular, the data that the prover sends at each round are the “one-dimensional” slices of this multivariate polynomial. In the first round, the prover sends the univariate polynomial

$$s_1(X) = \sum_{(x_2, \dots, x_n) \in \{0, 1\}^{n-1}} p(X, x_2, \dots, x_n).$$

If the original claim is correct, then $s_1(X)$ has degree at most d , and moreover $s_1(0) + s_1(1) = c$. The verifier checks precisely these two conditions, and rejects if either fails.

If both checks pass, the verifier samples a random challenge $r_1 \leftarrow \mathbb{F}$ and sends it to the prover. The point of this random challenge is to “pin down” the prover’s behavior at a point of the verifier’s choice: **if the prover has lied** about $s_1(X)$, then **with high probability (at most $d/|\mathbb{F}|$) the fake s_1 will disagree** with the true polynomial at $X = r_1$. If we set the finite field size to be sufficiently large, e.g., at least 128 bits for cryptographic security, then this probability is truly negligible.

After this first round of interaction, the prover and verifier have effectively reduced the problem to showing that

$$\sum_{(x_2, \dots, x_n) \in \{0, 1\}^{n-1}} p_1(x_2, \dots, x_n) = c_1,$$

where we define $p_1(x_2, \dots, x_n) := p(r_1, x_2, \dots, x_n)$ and $c_1 := s_1(r_1)$. That is, we have “bound” the first variable to r_1 and now reduce to a new sum-check instance in $n - 1$ variables. The protocol then repeats the same pattern on this new instance: in the second round, the prover sends a univariate polynomial

$$s_2(X) = \sum_{(x_3, \dots, x_n) \in \{0, 1\}^{n-2}} p_1(X, x_3, \dots, x_n),$$

the verifier again checks the degree and the equation $s_2(0) + s_2(1) = c_1$, samples a fresh random $r_2 \leftarrow \mathbb{F}$, sets the new claim $c_2 := s_2(r_2)$, and continue.

After n rounds, all variables have been fixed to random challenges $r_1, \dots, r_n \in \mathbb{F}$, and the verifier is left with a single claim of the form

$$p(r_1, \dots, r_n) = c_n$$

that it can check directly using its oracle access to p .

What properties does the sum-check protocol satisfy? The first is completeness. If the original sum claim is correct

NTEN

Need
um-Ch
vervie
rief In
fulilin
olynon
xisting
um-Ch
few Ide
atchin;
enefits
onclus
itation

and the prover follows the rules, then every round's check passes, and at the end we really do have $p(r_1, \dots, r_n) = c_n$, so the verifier accepts.

The more interesting property is **soundness**, which is about what happens when the original claim is actually false. In that case, no matter how a (possibly malicious) prover behaves, at some round it must send a polynomial that is not the “right” one. Two different low-degree polynomials over a field can only agree on a small fraction of inputs (a fact known as the Schwartz–Zippel lemma [5]); thus, a random challenge r_i will land on a point where they differ except with probability at most $d/|\mathbb{F}|$. Thus, even when the claim is wrong, a cheating prover can make the verifier accept only with negligible probability.

Brief Interlude on Multilinear Polynomials

So far, we have only described sum-check as a protocol on polynomials over finite fields. But how does this relate to any computation performed in the real world? The key bridge between the two is called the **arithmetization** of the computation: we first transform the computation trace—such as the values of all registers over time, or the sequence of memory accesses and intermediate constraint values—into polynomials, and then we use sum-check to reason about those polynomials instead of the original computation.

The arithmetization method used in sum-check-based proof systems is to turn these traces and constraint tables into **multilinear polynomials**. A multilinear polynomial is a multivariate polynomial where each variable appears with degree at most 1. For instance,

$$q(X_1, X_2, X_3) = 3X_1X_3 + 2X_2 + 5$$

is multilinear, while $X_1^2 + X_2$ is not. A key fact is that a multilinear polynomial in n variables is *uniquely* determined by its values on the 2^n points of the **Boolean hypercube** $\{0, 1\}^n$. This gives a natural encoding of a length- 2^n vector $\mathbf{v} = (v_0, \dots, v_{2^n-1})$ over the finite field $\mathbb{F}$.

We can also make this encoding precise with a mathematical formula. Given a function $p : \{0, 1\}^n \rightarrow \mathbb{F}$ (for example, a vector of trace or constraint values indexed by $y \in \{0, 1\}^n$), its **multilinear extension** is the *unique* multilinear polynomial $\tilde{p}(X_1, \dots, X_n)$ that agrees with p on all Boolean points. The formula for the multilinear extension is as follows:

$$\tilde{p}(X_1, \dots, X_n) = \sum_{y \in \{0, 1\}^n} \tilde{eq}(\vec{X}, y) \cdot p(y),$$

where the **equality polynomial** $\tilde{eq}$ is defined by

$$\tilde{eq}(\vec{X}, \vec{Y}) = \prod_{i=1}^n ((1 - X_i)(1 - Y_i) + X_i Y_i).$$

You should think of $\tilde{eq}$ as the indicator function of the equality relation on the Boolean hypercube. Indeed, for any $\vec{x}, \vec{y} \in \{0, 1\}^n$, we have $\tilde{eq}(\vec{x}, \vec{y}) = 1$ if $\vec{x} = \vec{y}$ and 0 otherwise. Plugging this into the multilinear extension formula, we can see that each term in the sum “picks out” the value $p(y)$ at exactly one point on the hypercube, thanks to the equality polynomial $\tilde{eq}(\vec{X}, y)$.

As a small example, the 2-variate multilinear extension is given by

$$\begin{aligned} p(X_1, X_2) &= p(0, 0) \cdot (1 - X_1)(1 - X_2) + p(0, 1) \cdot (1 - X_1)X_2 \\ &\quad + p(1, 0) \cdot X_1(1 - X_2) + p(1, 1) \cdot X_1X_2. \end{aligned}$$

NTEN

Need
um-Ch
vervie
rief In
fultilin
olynon
xisting
um-Ch
few Ide
atchin;
enefits
onclus
itation

Example: Arithmetizing a Batched Zero-Check

Now that we have seen multilinear polynomials, let's see how they allow us to turn common constraints into sum-check instances.

Consider a scenario where we have two columns of data, A and B , each of length $N = 4$. We want to **prove** that the element-wise product of these columns is zero. That is, if $A = [a_0, a_1, a_2, a_3]$ and $B = [b_0, b_1, b_2, b_3]$, we **claim** that

$$a_i \cdot b_i = 0 \quad \text{for all } i \in \{0, 1, 2, 3\}.$$

This **batched zero-check** pattern appears everywhere in proving program execution. For example, given two input registers holding values x and y , and an output register holding value z , we can enforce that all addition instructions are correctly performed by checking that the formula

$$S_{\text{ADD}}(i) \cdot (z(i) - (x(i) + y(i))) = 0$$

holds for all cycles i . Here, S_{ADD} is a selector that is 1 if the instruction is ADD and 0 **otherwise**.

To use sum-check, we view the indices as points on the boolean hypercube $\{0, 1\}^2$ and the columns as evaluations of multilinear polynomials. Let $p(x_1, x_2)$ and $q(x_1, x_2)$ be the multilinear extensions of A and B respectively. Then our claim becomes:

$$p(x_1, x_2) \cdot q(x_1, x_2) = 0 \quad \text{for all } (x_1, x_2) \in \{0, 1\}^2.$$

This pointwise condition can be bundled into a single sum over the hypercube. Define

$$g(x_1, x_2) = p(x_1, x_2) \cdot q(x_1, x_2),$$

and consider the claim that

$$g(x_1, x_2) = 0 \quad \text{for all } (x_1, x_2) \in \{0, 1\}^2.$$

This claim is equivalent to saying that the multilinear extension $\tilde{g}$ should vanish on all Boolean points. In other words,

$$\sum_{(x_1, x_2) \in \{0, 1\}^2} \tilde{g}((X_1, X_2), (x_1, x_2)) \cdot p(x_1, x_2) \cdot q(x_1, x_2) = 0$$

as multilinear polynomials. We now reduce this to a sum-check instance by sampling a random point $(r_1, r_2) \in \mathbb{F}^2$ and checking that the polynomial vanishes at this point:

$$\sum_{(x_1, x_2) \in \{0, 1\}^2} \tilde{e}\tilde{q}((r_1, r_2), (x_1, x_2)) \cdot p(x_1, x_2) \cdot q(x_1, x_2) = 0.$$

If this sum-check claim is correct, then the original multilinear polynomial $\tilde{g}$ is **identically zero**, except with negligible probability $2/|\mathbb{F}|$. So, instead of checking four separate equalities $g(x) = 0$ at the cube points, we check a **single** polynomial identity at a random point. Our exposition generalizes easily to an arbitrary 2^n -batched zero-check, which is converted into a n -variable sum-check instance.

Existing Algorithms for Sum-Check

We now turn to algorithms for the prover. We will focus on the most common setting in modern proof systems: sum-check over a low-degree function applied to one or more multilinear polynomials. This captures many real

NTEN

Need
um-Ch
wervie
rief In
fultilin
olynon
xisting
um-Ch
ew Ide
atchin;
enefits
onclus
itation

applications, such as the batched zero-check we saw earlier. To simplify the exposition, we further specialize to the case of a product of two multilinear polynomials

$$\sum_{x \in \{0,1\}^n} p(x) \cdot q(x) = c,$$

where $p, q : \{0,1\}^n \rightarrow \mathbb{F}$ are given by their evaluations on the Boolean hypercube. The algorithms we cover straightforwardly generalize to the product of arbitrary number of multilinear polynomials.

Linear-time algorithm

The first prover algorithm we consider, following Vu, Setty, Blumberg, and Walfish [6] and Thaler [7], runs in linear time in the number of summands (which is $N = 2^n$). This algorithm is key to the efficiency of sum-check, as we can prove a statement (like a batched zero-check) with only a constant-factor computational overhead.

Recall that in the first round of sum-check, the prover needs to compute and send the univariate polynomial

$$s_1(X) = \sum_{(x_2, \dots, x_n) \in \{0,1\}^{n-1}} p(X, x_2, \dots, x_n) \cdot q(X, x_2, \dots, x_n).$$

Since $s_1(X)$ has degree at most 2, it is completely determined by its evaluations on three distinct points, which we may choose to be $s_1(0)$, $s_1(1)$, and $s_1(2)$. The prover can compute these in a single pass over the 2^n evaluations of p and q . For each point $x' = (x_2, \dots, x_n)$, given evaluations of p and q at $(0, x')$ and $(1, x')$, the prover adds the following to the running sums $s_1(0)$, $s_1(1)$, $s_1(2)$:

$$p(0, x') \cdot q(0, x'), \quad p(1, x') \cdot q(1, x'), \quad p(2, x') \cdot q(2, x').$$

In the above, the last product can be computed by relying on the fact that p and q are linear in the first variable:

$$p(2, x') \cdot q(2, x') = (2 \cdot p(1, x') - p(0, x')) \cdot (2 \cdot q(1, x') - q(0, x')).$$

After the verifier sends a random challenge r_1 , the prover needs the “bound” polynomials

$$p_1(x_2, \dots, x_n) := p(r_1, x_2, \dots, x_n), \quad q_1(x_2, \dots, x_n) := q(r_1, x_2, \dots, x_n)$$

in order to continue the protocol on $n - 1$ variables. The linear-time algorithm explicitly materializes the evaluations of p_1 and q_1 on $\{0,1\}^{n-1}$ by making another pass over the original table and writing out two new vectors of evaluations, each of size 2^{n-1} . The update formula is

$$p_1(x_2, \dots, x_n) = (1 - r_1) \cdot p(0, x_2, \dots, x_n) + r_1 \cdot p(1, x_2, \dots, x_n),$$

$$q_1(x_2, \dots, x_n) = (1 - r_1) \cdot q(0, x_2, \dots, x_n) + r_1 \cdot q(1, x_2, \dots, x_n).$$

In each subsequent round $i = 2, 3, \dots, n$, the prover repeats the same pattern on these smaller vectors of evaluations to compute s_i , then binds the next challenge r_i and shrinks the vectors of evaluations again, and so on.

Cost Analysis: Given the description of the sum-check prover, it is clear that in round i , the prover needs to perform $O(2^{n-i})$ number of field operations (additions, subtractions, and multiplications). Summing over all rounds, the total work is therefore linear in the number of summands $N = 2^n$:

$$O(2^n + 2^{n-1} + \dots + 1) = O(2^n).$$

One downside of this algorithm is the need to keep **linear** storage (for instance, starting from round 2, where the prover needs to store p_1 and q_1). This linear-space usage may be fine for small-to-medium sum-check instances, but

ITEM

Need
um-Ch
vervie
rief In
fultilin
olynon
xisting
um-Ch
few Ide
atchin;
enefits
onclus
itation

eventually will become a bottleneck. Concretely, this means that on consumer hardware with (say) 16GB of RAM, we can prove a few dozen millions cycles of RISC-V program execution, but not a billion cycles. As RAM is a limited resource, this limitation motivates us to look for algorithms that use fewer memory, even at the cost of some extra recomputation.

Streaming algorithm with logarithmic space

We now present another classic sum-check proving algorithm, by [Cormode, Mitzenmacher, and Thaler](#) [8], that is much more memory efficient. The setting is the following. Assume there is enough persistent storage (for example, on disk) to hold all evaluations of p and q on $\{0, 1\}^n$, but not enough RAM to store the intermediate “bound” evaluation values of p_1 , q_1 , and so on. Alternatively, we can generate the original evaluations of p and q cheaply on the fly, but do not have space to cache all of the bound evaluations.

In this setting, the prover will need to perform a streaming pass over the original data in order to compute the univariate polynomial $s_i(X)$ for each round i . Recall that this polynomial is determined by its evaluations at $u \in \{0, 1, 2\}$:

$$s_i(u) = \sum_{(x_{i+1}, \dots, x_n) \in \{0, 1\}^{n-i}} p(r_1, \dots, r_{i-1}, X, x_{i+1}, \dots, x_n) \cdot q(r_1, \dots, r_{i-1}, X, x_{i+1}, \dots, x_n).$$

By the multilinear extension formula and the equality polynomial introduced earlier, we can write these partially bound evaluations as a further sum

$$p(r_1, \dots, r_{i-1}, u, x_{i+1}, \dots, x_n) = \sum_{y \in \{0, 1\}^{i-1}} \tilde{eq}((r_1, \dots, r_{i-1}), y) \cdot p(y, u, x_{i+1}, \dots, x_n),$$

and the same holds for q .

In other words, the value $p(r_1, \dots, r_{i-1}, u, x_{i+1}, \dots, x_n)$ is just a weighted sum of the original evaluations $p(y)$. Importantly, the weight for a given $p(y)$ depends only on the prefix of y (the first $i - 1$ bits) and the target evaluation point u .

This observation translates directly into a streaming algorithm. To compute the three required values $s_i(0)$, $s_i(1)$, $s_i(2)$ for round i , the prover can do the following:

1. **Initialize:** Set three accumulators for $s_i(0)$, $s_i(1)$, $s_i(2)$ to zero.
2. **Stream:** Read the original evaluation stream $p(x)$, $q(x)$ over all $x \in \{0, 1\}^n$, split into chunks of size 2^i that agrees on the suffix $x_{i+1}, \dots, x_n$.
3. **Update:** For each chunk, compute the appropriate weights based on the current round i and challenges $r_1, \dots, r_{i-1}$ for the prefix $y = (x_1, \dots, x_i)$. Importantly, the equality-polynomial weights can be computed on-the-fly in $O(2^i)$ time based on the challenges $r_1, \dots, r_{i-1}$. Add the weighted product terms to the respective accumulators.
4. **Interpolate:** Once the stream is finished, the accumulators hold $s_i(0)$, $s_i(1)$, $s_i(2)$. Use these to recover the polynomial $s_i(X)$.

This method never stores any intermediate “bound” tables. Instead, every round re-reads the original N values from the stream. This dramatically reduces the memory footprint to just $O(n)$ for storing the challenges and accumulators (or $O(\log N)$).

Cost Analysis:

- **Time:** We iterate over the full stream of size N for each of the n rounds. The total work is $O(n \cdot N) =$

NTEN

Need
um-Ch
ervie
rief In
fulilin
olynon
xisting
um-Ch
ew Ide
atchin;
enefits
onclus
itation

$O(N \log N)$, making this a **quasilinear-time** algorithm.

- **Space:** This is only $O(n) = O(\log N)$ since we only need to store the n challenges and the three accumulators over all rounds.

This $O(\log N)$ -space algorithm allows proving much larger statements than the linear-space approach, limited only by disk capacity or regeneration time rather than RAM. In practice, implementations often use a hybrid approach: stream for the first few rounds until the effective problem size shrinks enough to fit in memory (which happens before the halfway point in the protocol), then switch to the faster linear-time algorithm. However, this still leads to a large overhead compared to the linear-time algorithm, up to an order of magnitude for large instances (with $n \approx 30$).

New Idea: Round Batching and its Benefits

Both the linear-time and the **CMT** streaming algorithm share a common structure: they proceed **round-by-round**. For each round i , the prover computes a univariate polynomial $s_i(X)$, sends it, waits for the verifier's challenge r_i , and only then computes the polynomial $s_{i+1}(X)$ for the next round.

What if we could break this sequential dependency? Instead of computing just one round at a time, could the prover compute, say, rounds 1 and 2 simultaneously? Or any number of consecutive rounds at once?

This is the idea behind **round batching**, a technique that is recently introduced in my work [9] and the work of Bajewa et al. [10] (the latter is improved in another in-preparation work of mine). The idea turns out to bring efficiency gains for both algorithms above, especially in the setting of proving program execution. I will first discuss the mechanics of round batching, and then discuss the benefits in more detail.

At first glance, this proposal seems impossible: the prover cannot know the claim for round 2 without knowing the challenge r_1 from round 1. The key observation is that the prover can compute a response that is **oblivious** to the future challenges. This is achieved by computing a *bivariate* polynomial

$$s(X_1, X_2) = \sum_{x' \in \{0,1\}^{n-2}} p(X_1, X_2, x') \cdot q(X_1, X_2, x')$$

that suffices to answer both rounds 1 and 2. Indeed, the first round polynomial $s_1(X_1)$ is equal to $s(X_1, 0) + s(X_1, 1)$, and once the prover receives the first challenge r_1 , the second round polynomial $s_2(X_2)$ is equal to $s(r_1, X_2)$.

More generally, if we want to compute w consecutive rounds at once, starting at round i , the prover will need to compute the w -variate polynomial

$$s(X_1, \dots, X_w) = \sum_{x' \in \{0,1\}^{n-w-i}} p(r_1, \dots, r_{i-1}, X_1, \dots, X_w, x') \cdot q(r_1, \dots, r_{i-1}, X_1, \dots, X_w, x').$$

Once the prover has computed this polynomial, the next w rounds of the sum-check protocol reduces to doing sum-check over the w -variate polynomial s itself:

$$\sum_{x \in \{0,1\}^w} s(x) = c_i,$$

as we saw in our example above with $i = 1$ and $w = 2$.

Since w is often much smaller than n , this sum-check instance is trivial to prove. The main cost is in computing $s(X_1, \dots, X_w)$ itself. Since s is a **multi-quadratic** polynomial (having degree at most 2 in each variable), it is completely determined by its evaluations on the $\{0, 1, 2\}^w$ grid. For instance, if $w = 2$, we can compute one of

NTEN

Need
um-Ch
vervie
rief In
fulilin
olynon
xisting
um-Ch
few Ide
atchin;
enefits
onclus
itation

these evaluations (say $s(1, 2)$) as

$$\begin{aligned} s(1, 2) &= \sum_{x' \in \{0,1\}^{n-2}} p(r_1, \dots, r_{i-1}, 1, 2, x') \cdot q(r_1, \dots, r_{i-1}, 1, 2, x') \\ &= \sum_{x' \in \{0,1\}^{n-2}} (2 \cdot p(r_1, \dots, r_{i-1}, 1, 1, x') - p(r_1, \dots, r_{i-1}, 1, 0, x')) \\ &\quad \cdot (2 \cdot q(r_1, \dots, r_{i-1}, 1, 1, x') - q(r_1, \dots, r_{i-1}, 1, 0, x')). \end{aligned}$$

The apparent cost. At first glance, round batching seems to be strictly slower. In the round-by-round sum-check algorithms (whether linear-time or streaming), we only need to compute **3** evaluations of a quadratic polynomial every round (or $d + 1$ evaluations for a general sum-check instance over a degree- d product of multilinear). With round batching of window w , we need to compute 3^w (or $(d + 1)^w$ in the general case) evaluations over the w rounds. If we had computed round-by-round evaluations instead, this number is only $w \cdot (d + 1)$. The gap between $(d + 1)^w$ and $w \cdot (d + 1)$ grows very quickly - for instance, with $d = 2$ and $w = 3$, we are computing **three** times as many evaluations by batching three rounds together. Why do we even bother to explore this approach?

Why does round batching help?

The key insight is that the cost of computing evaluations is not equal across all rounds. In particular, there are two settings where round batching leads to a speedup despite the apparent extra work.

Small-field arithmetic at the start. In zkVMs, the underlying data (register values, memory contents) are small—typically 64-bit integers—while the proof system operates over a large, often 128-bit or even 256-bit field. Arithmetic with 64-bit integers are about **10-50x** faster compared to generic field multiplication.

For sum-check involving these small values, the prover enjoys a speedup in the first round, since the original evaluations

$$(p(x), q(x))_{x \in \{0,1\}^n}$$

are small. However, this speedup goes away in later rounds, since the bound evaluations

$$(p(r_1, \dots, r_{i-1}, x), q(r_1, \dots, r_{i-1}, x))_{x \in \{0,1\}^{n-i}}$$

are full field elements.

Round batching allows the prover to enjoy cheap multiplications in the first $w > 1$ rounds. Indeed, applying the technique to the first w rounds means that the prover needs to compute the evaluations

$$s(x_1, \dots, x_w) = \sum_{x' \in \{0,1\}^{n-w}} p(x_1, \dots, x_w, x') \cdot q(x_1, \dots, x_w, x').$$

Since the original evaluations

$$(p(x), q(x))_{x \in \{0,1\}^n}$$

are small, the extended evaluations

$$(p(x_1, \dots, x_w, x'), q(x_1, \dots, x_w, x'))_{(x_1, \dots, x_w) \in \{0,1,2\}^w, x' \in \{0,1\}^{n-w}}$$

are also small, since they are simple linear combinations of the original ones. Thus, round batching trades off expensive field multiplication that would have been incurred in rounds $2, \dots, w$, with a larger number of much

NTEN

Need
um-Ch
wervie
rief In
fultilin
olynon
xisting
um-Ch
few Ide
atching
enefits
onclus
itation

cheaper small multiplications. In practice, we can set $w \approx 3$ for about a $2 - 3\times$ speedup in prover time.

Fewer streaming passes. Recall that in the CMT streaming algorithm, the prover needs to make a pass over the original input in every round (or as long as we don't have space to store the bound evaluations). The cost of each pass is $O(N) = O(2^n)$, which *does not shrink* as the protocol progresses.

Round batching allows us to reduce the number of passes over the original inputs. Instead of a pass every round in a window of w consecutive rounds, round batching only requires a single streaming pass at the beginning of this window. The increase in evaluation cost is *smaller* as the rounds progress, costing only 2^{n-i} multiplication per evaluation if we batch at round i . Thus, we are trading off more evaluation computations, which scale as $(d+1)^w \cdot 2^{n-i}$, in lieu of a $(w-1) \cdot 2^n$ cost saved by doing fewer streams over the input. With the right schedule—small windows early when evaluation computation is more costly, and larger windows later (increasing exponentially)—we can improve the number of streaming passes **asymptotically** from $O(\log n)$ to $O(\log \log n)$. In practice, this can be as much as a $3 - 5\times$ speedup over the original CMT algorithm.

Conclusion

The sum-check protocol has become the workhorse of modern proof systems, powering the fastest zkVMs as well as a large number of sub-protocols such as polynomial commitment schemes [11]. In this post, we have seen the evolution of sum-check proving algorithms: from the classic linear-time prover that trades memory for speed, to the streaming algorithm that sacrifices time for a minimal memory footprint. Both approaches share a common limitation—they process rounds sequentially, binding each challenge before moving to the next.

We then saw how round batching breaks this sequential dependency, leading to two practical wins: it keeps arithmetic in fast, small-value operations for longer, and it reduces the number of streaming passes needed for memory-constrained settings. The technique is already being integrated into Jolt; without the small-value computation, for example, sum-check will be even more of a bottleneck than the current 70% of total prover time.

As zkVMs see broader adoption—from blockchain scalability to privacy-preserving identity—every percentage point of prover speedup translates to real cost savings and expanded applicability. The gap between native execution and proof generation is closing quickly, as algorithmic refinements like round batching and hardware acceleration continue to chip away at the overhead. It is perhaps surprising that we are still extracting efficiency from the sum-check protocol; despite being over three decades old, we continue to find new ways to optimize it for modern constraints.

Citations:

- [1] Google. Longfellow ZK: Implementation of the Google Zero-Knowledge library for Identity Protocols. GitHub repository. <https://github.com/google/longfellow-zk>. See also: Frigo, M., & Shelat, A. (2025). The Longfellow Zero-knowledge Scheme (draft-google-cfrg-libzk-01). Internet-Draft, IETF. <https://datatracker.ietf.org/doc/draft-google-cfrg-libzk/>
- [2] EthProofs. EthProofs: Zero-knowledge proofs resources for Ethereum. <https://ethproofs.org/>
- [3] a16z crypto. Jolt: a zkVM for general-purpose computation. GitHub repository. <https://github.com/a16z/jolt>
- [4] Lund, C., Fortnow, L., Karloff, H., & Nisan, N. (1992). Algebraic methods for interactive proof systems. Journal of the ACM, 39(4), 859–868. <https://dl.acm.org/doi/10.1145/146585.146605>
- [5] Schwartz, J. T. (1980). Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM, 27(4), 701–717.

NOTE

Need
um-Ch
vervie
rief In
fultilin
olynon
xisting
um-Ch
few Ide
atching;
enefits
onclus
itation

[6] Vu, V., Setty, S., Blumberg, A. J., & Walfish, M. (2013). A Hybrid Architecture for Verifiable Computation. In Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP), 223–237. IEEE.

[7] Thaler, J. (2013). Time-Optimal Interactive Proofs for Circuit Evaluation. In Advances in Cryptology – CRYPTO 2013 (LNCS 8042, pp. 71–89). Springer.

[8] Cormode, G., Mitzenmacher, M., & Thaler, J. (2012). Practical Verified Computation with Streaming Interactive Proofs. In Proceedings of the 2nd Innovations in Theoretical Computer Science Conference (ITCS 2012), 90–112. ACM.

[9] Bagad, S., Dao, Q., Domb, Y., & Thaler, J. (2025). Speeding Up Sum-Check Proving. Cryptology ePrint Archive, Paper 2025/1117. <https://eprint.iacr.org/2025/1117>

[10] Baweja, A., Chiesa, A., Fedele, E., Fenzi, G., Mishra, P., Mopuri, T., & Zitek-Estrada, A. (2025). Time-Space Trade-Offs for Sumcheck. In Theory of Cryptography Conference (TCC 2025).

[11] Arnon, G., Chiesa, A., Fenzi, G., & Yogev, E. (2024). WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification. Cryptology ePrint Archive, Paper 2024/1586. <https://eprint.iacr.org/2024/1586>

Need
um-Ch
vervie
rief In
fultilin
olynon
xisting
um-Ch
few Ide
atching
enefits
onclus
itation

Author: Quang Dao

Approved by: Elaine Shi, Sarah Scheffler, Harrison Grodin

#Security #Theory

#sumcheck #cryptography #proof systems #sumcheck proving #streaming algorithms

◀ Computer Assisted Mathematics: A Case Study in Discrete Geometry